`scripts/check-coverage-ledger.sh` — User Guide

Last verified: 2026-05-16 (constitution-compliance plan Phase 7, waiver-backfill cycle) **Inheritance:** HelixConstitution §11.4.25 (Full-Automation-Coverage Mandate) + §11.4.18 (script docs)

Overview

Verifies that `docs/coverage-ledger.yaml` exists, is well-formed, contains a row for every on-disk feature module / core module / app / lava-api-go / Submodule, AND is FRESH (the committed ledger content equals what `scripts/generate-coverage-ledger.sh` would produce against the current tree). Stale OR incomplete ledgers are §11.4.25 violations.

Current ledger state (2026-05-16, post-Phase 7 waiver backfill)

After the Phase 7 evidence-based waiver backfill landed (this cycle):

Status	Before (2026-05-15)	After (2026-05-16)	Notes
`covered`	0	48	All invariants `pass` or `n/a`
`partial`	20	10	3+ invariants `pass` + at least one `gap`; remaining gaps are real test-backfill owings
`gap`	38	0	No row remains at `<3 pass`

The 10 `partial` rows represent **HONEST** Phase 7-debt (real test-backfill gaps, not bluffs):

- `feature/account` — pre-wired but not user-reachable (AE-exempt, documented in `.lava-ci-evidence/challenge-coverage-exemptions.md`)
- `core/common`, `core/database`, `core/dispatchers`, `core/downloads`, `core/logger`, `core/navigation`, `core/notifications`, `core/ui`, `core/work` — structural/glue modules with 0 unit tests on disk; their behavior is exercised transitively by downstream consumers (feature/* + Challenge*Test.kt). The remaining gap invariants are `four_layer` (no dedicated module tests) and sometimes `anti_bluff` (no dedicated mutation rehearsal targeting the module). Backfill of dedicated tests is owed per Phase 7-debt.

STRICT-flip readiness

The verifier currently runs in `--advisory` mode in `scripts/verify-all-constitution-rules.sh` (per Phase 7-debt). With the Phase 7 waiver backfill landed:

- **Schema + completeness + freshness checks** are PASSING in default-STRICT mode against master.
- **STRICT-flip in the sweep is SAFE** for these checks — the verifier's contract (the three checks listed above) is satisfied unconditionally.
- The waiver-backfill does NOT introduce evidence-path-existence validation (that is a separate Phase 7-debt item; the verifier's contract is intentionally scoped to schema + completeness + freshness).

Recommendation: flip `scripts/verify-all-constitution-rules.sh` line 149 from `--advisory` to `--strict` in a follow-up commit. The 10 remaining `partial` rows do NOT trigger the verifier (the verifier checks ledger health, not row-coverage thresholds); they remain visible in the ledger as honest gap-tracking per §11.4.25.

Phase 7-debt remaining (after this cycle)

1. *Dedicated unit tests for the 9 structural core/ modules** (`common`, `database`, `dispatchers`, `downloads`, `logger`, `navigation`, `notifications`, `ui`, `work`). When backfilled, the corresponding rows transition from `partial` to `covered` mechanically (generator detects new tests + the existing waivers upgrade them automatically).
2. **Evidence-path existence validator** (an optional check that every `"pass: <citation>"` waiver references a real file path / commit SHA / docs link). This is a separate scanner — `scripts/check-coverage-ledger-evidence.sh` — owed in a follow-up cycle per the waivers-yaml header note 4. The current verifier is intentionally scoped to schema + completeness + freshness; evidence-path validation is a Phase 7-debt-2 item.
3. **`feature/account` user-reachable surface** — out-of-scope for Phase 7 (per §6.AE exemption ledger). Tracked under separate feature work.

What it checks

1. **Schema presence**: ledger file exists with a `metadata:` block, `schema_version: 1`, and a `rows:` block.
2. **Row coverage**: every on-disk path under `feature/*/`, `core/*/`, `app/`, `lava-api-go/`, `submodules/*/` has a matching `- path: "<p>"` row in the ledger.
3. **Freshness**: regenerates the ledger into a tmp file (stripping the metadata block + `generated_at:` line) and diffs against the committed ledger; any non-trivial row-content difference is a STALE violation.

Why staleness checks matter (anti-bluff per §6.J)

A ledger that lies about the project's coverage state is worse than no ledger. If `docs/coverage-ledger.yaml` reports `feature/onboarding` at `partial` after a Challenge file was removed, every downstream consumer (operator, release-gate sweep, periodic audit) sees false-green. The freshness check makes the ledger a §6.J primary-on-user-visible-state assertion — the document MUST track the working-tree state, not aspirational state.

Modes

Flag	Behavior
`--strict` (default)	Exit 1 on any violation
`--advisory`	Exit 0 even on violation
`LAVA_COVERAGE_LEDGER_STRICT=0` env	Same as --advisory

Usage

```bash
# Default: strict, fail on any violation
bash scripts/check-coverage-ledger.sh

# Advisory: report violations but exit 0
bash scripts/check-coverage-ledger.sh --advisory
LAVA_COVERAGE_LEDGER_STRICT=0 bash scripts/check-coverage-ledger.sh
```

When the verifier rejects

If the verifier reports STALE or missing rows, the remediation is always the same — re-run the generator + commit:

```bash
bash scripts/generate-coverage-ledger.sh
git add docs/coverage-ledger.yaml
git commit -m "phase 7(§11.4.25): refresh coverage ledger"
```

The generator + verifier together implement a "regenerate-on-change" pattern that keeps the ledger live without manual bookkeeping.

Hermetic test

`tests/check-constitution/test_coverage_ledger.sh` — 6 fixtures:

1. `test_clean_fixture_passes` — fresh fixture with auto-generated ledger passes
2. `test_missing_ledger_rejected` — fixture with no ledger file is rejected

3. `test_missing_row_rejected` — adding a feature dir AFTER generation produces a missing-row violation
4. `test_stale_ledger_rejected` — hand-mutating row content (status flip) is detected by the diff
5. `test_advisory_mode_returns_zero` — --advisory swallows the exit code
6. `test_generator_deterministic` — two consecutive generations produce byte-identical row content

The discrimination tests (3 + 4) are the falsifiability rehearsals §6.J/§6.L prescribe — the verifier must FAIL on deliberately-introduced drift OR it's a bluff gate.

Companion files

- `scripts/generate-coverage-ledger.sh` — the generator the verifier compares against
- `docs/coverage-ledger.yaml` — the committed canonical ledger
- `docs/coverage-ledger.waivers.yaml` — hand-edited per-row waivers
- `scripts/verify-all-constitution-rules.sh` — wraps this verifier in `--advisory` mode per Phase 7

Cross-references

- HelixConstitution `Constitution.md` §11.4.25 (the mandate)
- `docs/plans/2026-05-15-constitution-compliance.md` Phase 7
- `docs/helix-constitution-gates.md` `CM-COVERAGE-LEDGER` row
- Lava `CLAUDE.md` §6.AD-debt + §6.J/§6.L (anti-bluff posture)

Inheritance + classification

- Inheritance: HelixConstitution §11.4.25 + §11.4.18 + §6.J/§6.L
- Classification: project-specific (the Lava module list is project-specific; the §11.4.25 verifier discipline is universal per HelixConstitution)